

Analysis of 'When Malware is Packin' Heat; Limits of Machine Learning Classifiers Based on Static Analysis Features'

Thomas Turner
M14507785

Nathan Hafely
enter M number here

1 INTRODUCTION

With the rise of machine learning and artificial intelligence in recent years, many malware analysis experts have begun to research how these new tools can be used in their field. Many papers have begun to be published on the topic of using machine learning classifiers to distinguish between packed benign and packed malicious samples. The paper that we analyzed aims to verify the accuracy of these tests and see for themselves if machine learning can indeed be used to distinguish the maliciousness of packed samples based on static features alone [1].

Creating tools to automatically detect if a sample is malicious or not is very sought after in the malware field. Currently the best and most effective way to determine if a sample is malicious is to have an expert analyzing it using various static and dynamic analysis tools. This process entails placing the malware in an isolated environment and analyzing the static features such as imports, strings, and assembly code without running the code. Then the analyst will run debug the code to see any registry key changes, file system changes, or network connections the sample is attempting to make. This process can be very tedious and can take an expert a long time to complete.

There are also many online tools now such as VirusTotal that be used to automatically scan a sample to determine if it is malicious or not. But, this method also has many downsides. VirusTotal works by using signature-based detection methods to determine if a sample is malicious. So, if the signature of a sample is similar to a known malware, then VirusTotal will detect that the given sample is potentially malicious [1]. This technique is effective at detecting previously known malware, but will struggle on types of malware that it has never encountered before.

VirusTotal is also known to struggle with packed samples. The paper mentions a study that was conducted by Lakshman Nataraj. In this study, they collected a large number of benign Windows executables and then packed them with with four different common packers. Next, they submitted all of these samples to VirusTotal. From this experimnt, they found that 96.7 percent of the samples caused ten or more malicious detections on VirusTotal [1]. This shows that VirusTotal also uses packing as a strong indicator of maliciousness instead of the actual functionality of a sample.

Nowadays, many benign samples are also packed to protect the authors from getting their intellectual property stole [1]. Therefore, it is no longer the normal for exclusively malicious samples to be

packed. Figure 1 below shows the results of a brief experiment that the researchers conducted. In this experimnt, they collected a large number of samples from the real world and then analyzed them to see how many of them were packed based on each category of benign, malicious, and suspicious [1]. The results show that in the worst quarters around 50% of the benign samples that they collected were packed [1].

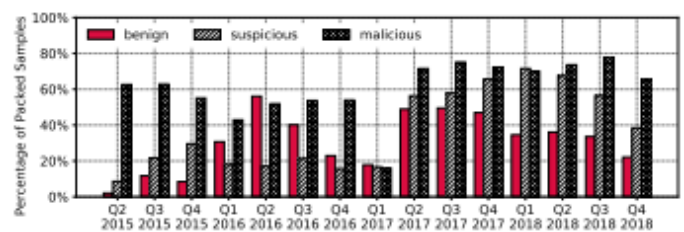


Figure 1: Prevalence of packed samples in the wild.

Through their experiments, the researchers show that packing routines do preserve some information on the function and maliciousness of a sample, but it is not enough to be applied to packers and packing routines that the classifier has never seen. Additionally, the classifier is susceptible to adversarial attacks [1]. This means that if malware authors can place benign code in their malware to try and confuse the classifier into thinking that the entire program is benign.

2 SYSTEM/THREAT MODEL

For their experiments, the researchers used samples exclusively from the Windows x86 operating system. The samples can be broken up into four unique categories: unpacked benign, packed benign, unpacked malicious, and packed malicious [1]. For each individual experiment, the data was broken up into a training set and a testing set. The type of classifier that was used for the analysis was a random forest model. Additionally, the classifier was trained on nine different sets of features in the samples (PE Headers, PE Sections, DLL Imports, API Imports, Rich Header, Byte n-grams, Opcode n-grams, Strings, File Generic) [1].

3 METHODOLOGY

This is the Methodology Section.

4 EXPERIMENT RESULTS AND ANALYSIS

This is the Experiment Results and Analysis Section.

5 CONCLUSION AND DISCUSSION

This is the Conclusion and Discussion Section.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ACM XXXX'18, XX 2018, XX, XX, XX
© 2018 Copyright held by the owner/author(s).
ACM ISBN 123-4567-24-567/08/06...\$15.00
https://doi.org/10.475/123_4

REFERENCES

- [1] Hojjat Aghakhani and Fabio Gritti and Francesco Mecca and Martina Lindorfer and Stefano Ortolani and Davide Balzarotti and Giovanni Vigna and Christipher Kruegel. 2020. When Malware is Packin' Heat; Limits of Machine Learning Classifiers Based on Static Analysis Features. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.